

Centro de Enseñanza Técnica Y Superior



Licea Sandoval Guillermo

ARQUITECTURA DE SOFTWARE

Tabla de olores dentro de las clases

Rafael Negrete Leyva (T035322)

Tijuana, B.C., 12 de mayo de 2026

Olores dentro de las clases

Categoría general	Subcategoría	Olores específicos incluidos	Qué revisa principalmente
Olores dentro de las clases	Olores medibles	Comentarios, método largo, clase larga, lista de parámetros larga	Problemas detectables con métricas o señales visibles: cantidad de líneas, cantidad de métodos, cantidad de variables, parámetros o comentarios.
Olores dentro de las clases	Nombres	Tipo incrustado en el nombre, nombre no comunicativo, nombres inconsistentes	Problemas de comunicación causados por nombres que no expresan intención, mezclan tipos o cambian para la misma idea.
Olores dentro de las clases	Complejidad innecesaria	Código muerto, generalidad especulativa	Código más complicado de lo necesario por historia, requisitos antiguos o diseño pensado para futuros usos que no llegaron.
Olores dentro de las clases	Duplicidad	Número mágico, código duplicado, clases alternativas con diferentes interfaces	Repetición de valores, lógica, estructuras o clases parecidas que aumentan mantenimiento y errores.
Olores dentro de las clases	Lógica condicional	Verificación de nulos, expresiones booleanas complicadas, caso especial, herencia simulada	Condicionales difíciles de razonar o usados como sustituto de mecanismos de orientación a objetos.

Olor específico	Breve descripción	Síntomas	Causas	Refactoring sugerido
Comentarios	El código necesita comentarios para explicar algo que debería comunicarse mejor con nombres, métodos o estructura.	Aparecen comentarios como <code>//</code> o <code>/* */</code> dentro del código, el comentario explica bloques, métodos o precondiciones.	El autor nota que algo no es claro y agrega una explicación. Algunos comentarios sí son útiles cuando explican el porqué o citan algoritmos no obvios.	Extract Method si el comentario explica un bloque, Rename Method si el comentario explica mejor el propósito del método, Introduce Assertion si explica precondiciones.
Método largo	Un método concentra demasiados pasos o responsabilidades, lo que dificulta leerlo y modificarlo.	Número grande de líneas, bloques separados por comentarios o líneas en blanco, varias tareas dentro del mismo método.	El método inicia con una trayectoria y se le agregan funciones o casos sin separar responsabilidades ni crear apoyos.	Extract Method para dividirlo en piezas pequeñas y significativas, evitando extraer métodos sin intención clara.
Clase larga	Una clase concentra demasiadas responsabilidades o información, por lo que se vuelve difícil de entender y mantener.	Muchas variables de instancia, muchos métodos o muchas líneas en una sola clase.	La clase crece poco a poco al agregar características, también puede faltar comprensión sobre sus partes internas.	Primero corregir métodos largos si existen. Después aplicar Extract Class, Extract Subclass o Extract Interface según la responsabilidad identificada.
Lista de parámetros larga	Un método necesita demasiada información externa para funcionar, lo que complica su uso y lectura.	Método con más de dos parámetros o con grupos de parámetros relacionados.	Se intenta reducir acoplamiento pasando todo desde el invocador, o se generaliza una rutina para demasiadas variaciones.	Replace Parameter with Method si el dato puede obtenerse de un objeto conocido, Preserve Whole Object si los datos vienen del mismo objeto, Introduce Parameter Object si forman un grupo lógico.
Tipo incrustado en el nombre	El nombre incluye el tipo de dato o estructura en vez de comunicar intención.	Nombres compuestos con el tipo del argumento, notación húngara como <code>iCount</code> , nombres que reflejan tipo y no propósito.	El desarrollador intenta hacer visible el tipo dentro del nombre, pero esto vuelve el nombre más técnico y menos expresivo.	Rename Method, Rename Field, Rename Variable o Rename Constant usando nombres que comuniquen intención sin involucrar tipos.
Nombre no comunicativo	El nombre no permite entender claramente qué representa o qué hace el elemento.	Nombres de uno o dos caracteres, nombres sin vocales, variables numeradas, abreviaciones o nombres equivocados.	Al implementar por primera vez se elige un nombre rápido, después aparece una mejor comprensión, pero el nombre no se actualiza.	Rename Method, Rename Field, Rename Variable o Rename Constant para expresar intención. En alcances cortos pueden aceptarse <code>i</code> , <code>j</code> , <code>k</code> o <code>c</code> si no confunden.
Nombres inconsistentes	La misma idea recibe nombres diferentes en distintas partes del sistema.	Un concepto, acción o dato aparece nombrado de manera distinta en varios métodos, clases o componentes.	Las clases o métodos fueron creados por diferentes personas o en distintos momentos sin un vocabulario común.	Elegir el mejor nombre y aplicar Rename de forma consistente para que las mismas cosas se llamen igual.
Código muerto	Código que ya no aporta comportamiento real al sistema y solo aumenta ruido.	Variable, parámetro, fragmento, método o clase que no se usa en ningún lado.	Cambios de requisitos, nuevos enfoques sin limpieza o lógica complicada que deja condiciones imposibles.	Remove Dead Code: borrar el código no utilizado y sus pruebas asociadas cuando ya no aplican.

Generalidad especulativa	Diseño agregado para necesidades futuras que no existen en los requisitos actuales.	Clases, métodos, campos o parámetros sin clientes, código más complicado de lo necesario.	Se construye código con la esperanza de que se vuelva útil después, pero ese uso nunca llega.	Inline Class si la clase es innecesaria, Inline Method o borrar método, borrar campos sin referencias, Remove Parameter para parámetros innecesarios.
Número mágico	Un valor constante aparece directamente en el código sin explicar su significado.	Constantes numéricas escritas en el cuerpo del código, por ejemplo límites, porcentajes o tamaños.	Alguien necesita un valor y lo coloca directamente en el código sin nombrarlo.	Replace Magic Number with Symbolic Constant para sustituir el número por una constante con nombre.
Código duplicado	Dos fragmentos se parecen mucho o producen casi el mismo efecto, aunque no sean idénticos.	Código visualmente casi igual, lógica repetida, errores que deben corregirse en varios lugares.	Programadores trabajan por separado sin notar código similar, o se copia y pega código con cambios menores.	Extract Method si está en la misma clase, Template Method si está en clases relacionadas, Extract Class si está en clases no relacionadas, crear algoritmo común si tienen el mismo efecto.
Clases alternativas con diferentes interfaces	Dos clases realizan tareas parecidas, pero con nombres de métodos o protocolos diferentes.	Clases que parecen hacer lo mismo, pero exponen operaciones con nombres, parámetros o interfaces distintas.	Se crea código similar para situaciones similares sin saber que ya existía otra solución.	Rename Method para igualar nombres, Move Method y agregar parámetros para unificar protocolos, Extract Superclass si son similares, eliminar la clase extra si es posible.
Verificación de nulos	El flujo depende repetidamente de revisar si un objeto es null.	Aparece muchas veces código como if (xxx == null).	Se usa null para representar un valor por omisión o ausencia, lo cual obliga a revisar nulos en varias partes.	Usar un valor por defecto si existe, de lo contrario aplicar Introduce Null Object con comportamiento seguro.
Expresiones booleanas complicadas	Condiciones difíciles de leer porque combinan demasiados operadores lógicos.	Condiciones con muchos AND, OR o NOT, expresiones anidadas o difíciles de negar.	La condición fue compleja desde el inicio o se le agregaron reglas con el tiempo.	Simplificar con leyes de De Morgan, renombrar variables para comunicar mejor, extraer la condición a un método con nombre claro.
Caso especial	El código agrega excepciones o caminos particulares antes de resolver la tarea general.	If complejos, verificaciones de valores específicos antes de hacer una operación.	Se detecta una situación especial y se agrega una condición en lugar de rediseñar el flujo general.	Replace Conditional with Polymorphism si el condicional sustituye polimorfismo, si if y else son parecidos, reescribir para usar un fragmento común.
Herencia simulada	El código decide el comportamiento revisando tipos o códigos en vez de usar clases y polimorfismo.	Switch, varios if en fila, instanceof o equivalentes para decidir con qué tipo trabajar.	Se evita crear nuevas clases o usar mecanismos de orientación a objetos del lenguaje.	Extract Method y Move Method si hace falta, Replace Type Code with Subclasses, State o Strategy, Replace Conditional with Polymorphism.